



## 2.1.1 Kafka + 3가지 대안 비교하기

|         |    |
|---------|----|
| ☼ 상태    | 완료 |
| ☰ 다중 선택 |    |

### ▼ **결정!!! Apache Kafka (실시간 대용량 스트리밍의 사실상 표준 / 엔터프라이즈급)**

쿠팡, 넷플릭스, 토스 등 대용량 실시간 처리가 필요한 모든 IT 기업이 사용하는 끝판왕 데이터 파이프라인

- **동작 방식:** 백엔드 서버(Producer)가 이벤트 로그를 Kafka의 'Topic'이라는 공간에 쓰면, 이 데이터를 필요로 하는 **여러 시스템(Consumer - 추천 엔진, 통계 서버, S3 백업 워커 등)**이 각자의 처리 속도에 맞춰 독립적으로 실시간 데이터를 가져갑니다.
- **장점:**
  1. **압도적인 처리량:** 초당 수십만~수백만 건의 트래픽이 몰려도 서버가 뻘지 않고 든든하게 방어(Buffer)해 줍니다.
  2. **완벽한 확장성(Pub/Sub):** 한 번 수집한 데이터를 A팀, B팀, C팀이 동시에 가져가도 시스템에 무리가 없습니다.
  3. **데이터 복구(Replay):** Redis와 달리 데이터를 하드디스크에 안전하게 보관하므로, 추천 엔진 서버가 잠깐 죽었다가 살아나도 못 받은 데이터부터 다시 읽어올 수 있습니다.
- **단점:** 설정, 운영, 유지보수 난이도가 극악에 가깝습니다.
  - 제대로 띄우려면 서버가 최소 3대(클러스터) 이상 필요하고
  - 메모리도 많이 먹기 때문에,
  - 소규모 프로젝트나 1인 개발에서는 인프라 관리하다가 지칠 수 있습니다.

### ▼ **대안 A - Redis Streams (가볍고 빠른 인메모리 방식 / 소규모 실시간 추천)**

Kafka의 무거운 인프라가 부담스러울 때 현업에서 가장 많이 선택하는 '가성비 실시간 파이프라인'입니다.

- **동작 방식:** 백엔드 서버가 로그 데이터를 Redis 메모리에 밀어 넣습니다( **XADD** ). **추천 엔진은 이 Stream을 실시간으로 구독(Subscribe)**하고 있다가 0.1초 만에 데이터를 가져가서 추천 결과를 업데이트합니다.
- **장점:** Kafka에 비해 설치와 설정이 압도적으로 쉽고 가볍습니다. 데이터를 하드디스크가 아닌 메모리(RAM)에서 처리하므로 처리 속도가 극단적으로 빠릅니다.
- **단점:** 메모리 용량의 한계가 있기 때문에 데이터를 오래 보관할 수 없습니다. 따라서 실시간 처리를 끝낸 데이터는 주기적으로 지워주거나 **S3로 백업하는 별도의 로직이 필요합니다.**

### ▼ **대안 B - AWS Kinesis Data Streams (클라우드 완전 관리형 / AWS 친화적 환경)**

클라우드 인프라에 익숙하고 서버 관리에 시간을 쏟고 싶지 않을 때 선택하는 완벽한 대체재입니다.

- **동작 방식:** 백엔드 서버가 Kafka 대신 **AWS Kinesis API로 로그를 쏩니다.** Kinesis가 알아서 스트림을 만들고, **추천 엔진(Consumer)은 이 스트림에서 실시간으로 데이터를 읽어갑니다.**
- **장점:** 직접 서버를 띄우고 관리할 필요가 없는 완전 관리형(Serverless)입니다.
  - Kinesis Firehose를 클릭 한 번으로 연결해 **실시간 처리는 Kinesis Streams가,**
  - **S3 백업은 Firehose가 동시에 알아서 처리**하게 만들 수 있습니다.
- **단점:** AWS 환경에 종속되며, 처리하는 데이터양과 켜두는 시간만큼 지속적으로 클라우드 비용이 발생합니다.

### ▼ 대안 C - RabbitMQ (정교한 라우팅이 필요한 전통적 메시지 브로커)

대용량 로그 수집보다는, 마이크로서비스 간에 복잡한 규칙으로 실시간 메시지를 주고받아야 할 때 유리합니다.

- **동작 방식:** 백엔드가 이벤트를 중앙(Exchange)으로 보내면, 미리 설정한 규칙에 따라 여러 개의 큐(Queue)로 데이터가 똑똑하게 분배되고 추천 엔진이 이를 가져갑니다.
- **장점:** "A 상품 로그는 1번 큐로, B 상품 로그는 2번 큐로 보내라" 같은 정교한 라우팅 규칙을 짜는 데 있어서는 Kafka보다 훨씬 유연하고 강력합니다.
- **단점:** 트래픽이 초당 수만 건을 넘어가는 극단적인 대용량 처리에서는 Kafka보다 성능이 떨어집니다. 또한, 이미 가져간 데이터를 나중에 처음부터 다시 읽어보는 기능(Replay)이 제한적입니다.

| 항목 \ 대안                      | Kafka                          | Redis Streams(탈락)                                 | AWS Kinesis (탈락)                | RabbitMQ (탈락)                                       |
|------------------------------|--------------------------------|---------------------------------------------------|---------------------------------|-----------------------------------------------------|
| S3 백업 + 추천엔진 동시 전송 (Pub/Sub) | 최상<br>(각자 완벽히 독립적으로 가져감)       | 보통<br>(가능은 하나 S3용 코드를 직접 짜야 함)                    | 최상<br>(Firehose가 S3 백업을 알아서 해줌) | 우수<br>(라우팅은 좋으나 데이터 보관에 취약)                         |
| 추천 모델 서버가 죽었을 때 (데이터 복구)     | 가능<br>(못 읽은 과거 데이터부터 다시 읽기 가능) | 제한적<br>(메모리 꽉 차서 지워지면 복구 불가)<br>→ <b>redis 탈락</b> | 가능<br>(과거 데이터 Replay 지원)        | 어려움<br>(가져간 데이터는 보통 바로 사라짐)<br>→ <b>RabbitMQ 탈락</b> |
| 설정 및 관리 난이도                  | 극상 (가장 어려움)                    | 하 (아주 쉬움)                                         | 중<br>(AWS 콘솔 세팅 필요)             | 중<br>(세팅 자체는 무난함)                                   |
| 비용 (클라우드 이전 시)               | 높음 (서버 여러 대 필요)                | 중간 (메모리 인스턴스 비용)                                  | 사용한 만큼만 청구 (유연함)                | 중간                                                  |
| 포트폴리오 매력도 (이력서)              |                                |                                                   |                                 |                                                     |

→ 다음의 이유로 **AWS Kinesis 탈락**

## 특정 클라우드 종속 방지 (Vendor Lock-in)

▼ Kinesis는 AWS가 독자적으로 만든 '완전 관리형 독점 서비스'이기 때문에,

Kinesis를 쓰면 데이터 파이프라인 전체가 AWS 생태계에 묶이게 된다

웹 서버를 AWS에 둔다고 해서 데이터 파이프라인 전체가 AWS 생태계에 묶이는 것은 절대 아닙니다. 이것이 현업 아키텍트들이 클라우드를 설계할 때 가장 중요하게 생각하는 '종속성(Lock-in)의 수준 차이'입니다. 왜 웹 서버는 괜찮고 Kinesis는 묶이는 것인지, 아주 쉽게 비교해 드릴게요.

### 1. 웹 서버 (AWS EC2) → "빈 사무실만 빌리는 것" (종속성 낮음)

유저님이 만든 Node.js 웹 서버 코드를 AWS의 가상 컴퓨터(EC2)에 올려서 돌린다고 가정해 보겠습니다.

- **상황:** AWS 요금이 갑자기 너무 비싸져서 구글 클라우드(GCP)로 이사 가고 싶어졌습니다.
- **결과:** 유저님의 코드는 순수한 Node.js 코드(또는 Docker 컨테이너)이기 때문에, 그냥 그 코드를 그대로 복사해서 구글 클라우드의 가상 컴퓨터에 붙여넣기만 하면 바로 돌아갑니다. 단 한 줄의 코드 수정도 필요 없습니다. 언제든 미련 없이 AWS를 떠날 수 있습니다.

### 2. Kafka를 AWS EC2에 직접 설치 → "내 가구를 들고 다니는 것" (종속성 낮음)

앞서 말씀드린 대기업들이 선호하는 방식입니다. AWS의 빈 컴퓨터(EC2)를 빌린 뒤, 그 위에 오픈소스인 Kafka를 직접 설치해서 씁니다.

- **상황:** 구글 클라우드로 이사 가고 싶습니다.
- **결과:** 구글 클라우드 컴퓨터를 빌려서 똑같이 Kafka를 설치하고 세팅만 옮기면 끝입니다. 파이프라인 구조가 그대로 유지됩니다.

### 3. AWS Kinesis 사용 → "책상이 바닥에 박혀있는 풀옵션 사무실" (종속성 높음)

Kinesis는 AWS가 독자적으로 만든 '완전 관리형 독점 서비스'입니다.

- **상황:** Kinesis의 편안함에 취해 백엔드 서버에서 로그를 보낼 때 Kinesis 전용 코드(`kinesis.putRecord(...)`)를 잔뜩 짜두었고, S3로 자동 백업되는 Firehose까지 예쁘게 연결해 두었습니다. 그런데 요금이 너무 나와서 구글 클라우드로 이사 가고 싶습니다.
- **결과: 대참사가 일어납니다.** 구글 클라우드에는 Kinesis도 없고 Firehose도 없습니다.
  1. 백엔드 서버에 짜둔 Kinesis 전용 전송 코드를 다 지우고, 구글 클라우드 전용(Pub/Sub)으로 코드를 전부 다시 짜야 합니다.
  2. 클릭 한 번으로 S3에 백업하던 Firehose가 없으니, 백업 파이프라인을 처음부터 새로 설계하고 구축해야 합니다.

### 💡 결론: 껌데기만 빌리느냐, 뼈대까지 빌리느냐

클라우드 회사(AWS, 구글, MS)들은 고객이 자기들 서비스를 영원히 쓰게 만들고 싶어 합니다. 그래서 웹 서버 같은 단순한 '가상 컴퓨터'를 넘어서, Kinesis, DynamoDB, Lambda 같은 '자기들 클라우드에서만 돌아가는 압도적으로 편한 독점 서비스'를 미친 듯이 만들어냅니다.

편리함을 취할수록 AWS라는 늪에 깊게 빠지게(Lock-in) 되고, 나중에 빠져나오려면(마이그레이션) 수개월의 시간과 수억 원의 개발 비용을 치러야 합니다.

그래서 실무에서는 핵심 파이프라인과 인프라(Kafka, DB 등)를 특정 클라우드에 종속되지 않게 설계하는 '**클라우드 아그노스틱(Cloud-Agnostic)**' 전략을 매우 중요하게 생각합니다. 유저님의 질문이 바로 이 거대한 IT 산업의 딜레마를 정확히 관통한 것입니다!

- 실무에서는 "웹 서버는 AWS, 데이터 분석과 AI는 구글 클라우드(GCP), 사내 보안 데이터는 자체 서버(On-premise)"를 섞어 쓰는 **멀티 클라우드(Multi-Cloud)** 전략을 많이 취합니다.
- **Kafka는 오픈소스**이기 때문에 AWS, GCP, 자체 서버 등 어디서든 똑같이 띄워서 쓸 수 있습니다. 회사가 AWS에서 다른 클라우드로 이사를 가더라도 파이프라인을 그대로 들고 갈 수 있는 강력한 자유도가 있습니다.

## 고민 과정)

파이프라인에서 RabbitMQ와 Redis는 S3 백업이나 장애 복구 측면에서 조금 아쉬운 부분들이 있다.

따라서 Kafka와 Kinesis 둘 중 하나를 선택하시는 것을 강력히 추천합니다. (나도동의함)

### 추천 1. "나는 데이터 파이프라인과 인프라를 다루는 역량을 어필하고 싶다" → Kafka

- 이력서나 포트폴리오에 "Kafka를 이용한 실시간 추천 파이프라인 구축"이라고 적히는 순간 파괴력이 다릅니다.
- **해결책:** 앞서 "서버가 3대 필요하다"고 했지만, 로컬 개발 환경에서는 **Docker Compose**를 사용하면 명령어 한 줄로 Kafka 환경을 띄울 수 있습니다. (실제 운영 환경이 아니라면 1대로도 테스트는 충분히 가능합니다.)
- **흐름:** 시뮬레이터 → **Kafka Topic** → (파이썬 추천 모델) & (S3 전송용 Node.js 워커)

### 추천 2. "나는 복잡한 인프라 세팅은 피하고, 추천 모델링과 비즈니스 로직에 집중하고 싶다" → AWS Kinesis

- 로컬 노트북의 한계를 가장 빨리 벗어날 수 있는 방법입니다. 파이프라인 자체를 AWS에 맡겨버리고, 로컬 컴퓨터에서는 가벼운 시뮬레이터와 웹 서버만 돌리면 됩니다.
- **해결책:** AWS 콘솔에서 Kinesis Streams(추천 모델용)와 Firehose(S3 백업용)를 클릭 몇 번으로 만듭니다.
- **흐름:** 시뮬레이터 → **AWS Kinesis** → 알아서 S3에 저장 & 파이썬 추천 모델로 실시간 전송

데이터를 생성하는 역할(웹+시뮬레이터)과 소비하는 역할(추천 엔진+S3)이 명확히 나뉘어 있어서 어떤 선택을 하든 아주 멋진 프로젝트가 될 것 같습니다.

## Q1. AWS Kinesis를 쓰면 어떤 비즈니스 로직에 집중할 수 있는가?

AWS Kinesis와 같은 완전 관리형(Serverless) 스트리밍 서비스를 도입하면,

인프라의 설치·운영·장애 대응(예: 서버 다운, 디스크 용량 관리, 클러스터 동기화 등)에 드는 시간을 제로(0)에 가깝게 줄일 수 있습니다.

→ 그 결과 개발자는 서비스의 핵심 가치를 만드는 **비즈니스 로직**과 데이터를 활용하는 다음 단계 파이프라인(Downstream Pipelines)에 온전히 모든 에너지를 쏟을 수 있게 됩니다.

구체적으로 어떤 부분들에 집중할 수 있게 되는지

### 1. 집중할 수 있는 핵심 '비즈니스 로직'

인프라 관리를 AWS에 맡기면, 실시간으로 쏟아지는 웹 이벤트와 시뮬레이터 데이터를 '어떻게 요리할 것인가'에 대한 규칙(비즈니스 로직)에만 집중할 수 있습니다.

- **실시간 실시간 점수 산정 알고리즘 고도화:** 유저가 특정 상품을 클릭하거나 장바구니에 담았을 때, 해당 카테고리에 대한 관심도 점수를 실시간으로 몇 점 부여할 것인지, 그리고 시간 경과에 따라 이 점수를 어떻게 감쇄(Decay)시킬 것인지에 대한 세부 규칙을 정교하게 짤 수 있습니다.
- **시뮬레이터 노이즈 및 어뷰징 필터링:** 테스트용 시뮬레이터에서 봇(Bot)처럼 생성되는 무의미한 대량의 클릭 데이터와 실제 웹 유저의 자연스러운 행동 데이터를 분리하고, 이상 패턴(예: 0.1초 만에 상품 10개 클릭)을 실시간으로 감지하여 추천 엔진에서 제외하는 필터링 로직에 집중할 수 있습니다.
- **실시간 개인화 세션 관리:** 로그인한 유저뿐만 아니라, 가입하지 않은 게스트 유저가 들어왔을 때 브라우저 세션 동안 발생한 몇 개의 클릭만으로 즉시 메인 화면 카테고리를 변경해 주는 단기 세션 추천 규칙을 개발할 수 있습니다.

### 2. '그 다음 파이프라인'에 더 집중할 수 있다.

Kinesis를 쓰면 데이터 수집(Ingestion) 단계가 완전히 자동화되기 때문에, 데이터를 정제하고 가공하여 최종 목적지까지 배달하는 **후속 파이프라인 구축에 압도적으로 많은 시간을 투자할 수** 있습니다.

- **S3 백업 파이프라인 제로 코딩 (Kinesis Data Firehose 활용):**

원래라면 메시지 큐에서 데이터를 하나씩 꺼내와 파일로 묶고 S3 API를 호출하는 백업 코드를 직접 구현해야 합니다. 하지만 Kinesis를 쓰면 **Kinesis Data Firehose**를 마우스 클릭 몇 번으로 붙여서 "1분마다 혹은 5MB마다 S3에 자동 저장"하도록 세팅할 수 있습니다. 백업 파이프라인 코딩 시간이 0이 됩니다.

- **실시간 가공 및 변환(Transform) 파이프라인:**

수집된 원본(Raw) 로그 데이터를 실시간 추천 모델이 바로 읽을 수 있는 형태(Feature)로 변환하는 파이프라인에 집중할 수 있습니다. 예를 들어, **AWS Lambda** 나 **Kinesis Data Analytics** 를 연결하여 스트림 내부의 데이터를 실시간으로 정제(필요 없는 필드 제거, 데이터 포맷 통일)하는 라인을 매끄럽게 구축할 수 있습니다.

- **추천 모델 서빙 및 피드백 루프:**

가공된 스트리밍 데이터를 파이썬 기반의 추천 모델 서버로 안전하게 전달하고, 모델이 내놓은 예측 결과를 다시 프론트엔드 웹 화면에 1~2초 만에 반영하는 '서빙 파이프라인'의 안정성을 높이는 데 집중할 수 있습니다.

- **S3 데이터 분석 파이프라인 (Athena 연동):**

Firehose를 통해 S3에 차곡차곡 쌓인 대용량 로그 파일들을 **AWS Athena**를 이용해 SQL 쿼리로 조회하고, 시뮬레이터 데이터의 분포를 분석하거나 대시보드를 만들어 추천 모델을 지속적으로 업데이트(재학습)하는 배치 파이프라인을 구축할 여유가 생깁니다.

아키텍처 선택(직접 구축하는 Kafka vs 완전 관리형 AWS Kinesis)...

## Q2. AWS Kinesis에는 비즈니스 로직에 집중할 수 있는 장점이 있음에도 왜 실무에서 Kafka를 더 많이 사용하고 더 표준이라고 여기는가?

"관리가 편하고 비즈니스 로직에 집중할 수 있는데 왜 대기업들은 굳이 사서 고생(Kafka)을 할까?"라는 의문은 시니어 아키텍트들도 클라우드 전환 시에 항상 토론하는 핵심 주제

비즈니스 로직에 집중할 수 있다는 엄청난 장점에도 불구하고, 실무(특히 중견~대기업 이상)에서 **Kafka를 압도적인 표준으로 사용하는 이유는** 크게 4가지 현실적인 문제 때문

### 1. 트래픽이 커질수록 폭발하는 '비용 (Cost at Scale)'

가장 결정적인 이유입니다. AWS Kinesis 같은 완전 관리형 서비스는 초기에는 비용이 저렴하지만, **데이터 전송량과 처리량이 늘어날수록 요금이 기하급수적으로 증가**합니다.

- **초기/스타트업:** "서버 관리자 1명 채용할 인건비(연 8천만 원)보다 Kinesis 한 달 요금(10만 원)이 싸다!" → **Kinesis 승리**
- **대기업 (토스, 넷플릭스 급):** 하루에 수백 테라바이트의 로그가 쏟아집니다. 이걸 Kinesis로 처리하면 한 달 요금이 수천만 원~수억 원이 나옵니다. 반면, 자체 서버(EC2나 온프레미스)에 Kafka 클러스터를 구축하면 인건비를 포함해도 비용이 훨씬 저렴해집니다.

### 2. 특정 클라우드 종속 방지 (Vendor Lock-in)

▼ Kinesis는 AWS가 독자적으로 만든 '완전 관리형 독점 서비스'이기 때문에,  
Kinesis를 쓰면 데이터 파이프라인 전체가 AWS 생태계에 묶이게 된다

웹 서버를 **AWS에 둔다고 해서 데이터 파이프라인 전체가 AWS 생태계에 묶이는 것은 절대 아닙니다.** 이것이 현업 아키텍트들이 클라우드를 설계할 때 가장 중요하게 생각하는 '종속성(Lock-in)의 수준 차이'입니다. 왜 웹 서버는 괜찮고 Kinesis는 묶이는 것인지, 아주 쉽게 비교해 드릴게요.

#### 1. 웹 서버 (AWS EC2) → "빈 사무실만 빌리는 것" (종속성 낮음)

유저님이 만든 Node.js 웹 서버 코드를 AWS의 가상 컴퓨터(EC2)에 올려서 돌린다고 가정해 보겠습니다.

- **상황:** AWS 요금이 갑자기 너무 비싸져서 구글 클라우드(GCP)로 이사 가고 싶어졌습니다.

- **결과:** 고객님의 코드는 순수한 Node.js 코드(또는 Docker 컨테이너)이기 때문에, 그냥 그 코드를 그대로 복사해서 구글 클라우드의 가상 컴퓨터에 붙여넣기만 하면 바로 돌아갑니다. **단 한 줄의 코드 수정도 필요 없습니다.** 언제든 미련 없이 AWS를 떠날 수 있습니다.

## 2. Kafka를 AWS EC2에 직접 설치 → "내 가구를 들고 다니는 것" (종속성 낮음)

앞서 말씀드린 대기업들이 선호하는 방식입니다. AWS의 빈 컴퓨터(EC2)를 빌린 뒤, 그 위에 오픈소스인 Kafka를 직접 설치해서 씁니다.

- **상황:** 구글 클라우드로 이사 가고 싶습니다.
- **결과:** 구글 클라우드 컴퓨터를 빌려서 똑같이 Kafka를 설치하고 세팅만 옮기면 끝입니다. 파이프라인 구조가 그대로 유지됩니다.

## 3. AWS Kinesis 사용 → "책상이 바닥에 박혀있는 풀옵션 사무실" (종속성 높음)

Kinesis는 AWS가 독자적으로 만든 '완전 관리형 독점 서비스'입니다.

- **상황:** Kinesis의 편안함에 취해 백엔드 서버에서 로그를 보낼 때 Kinesis 전용 코드( `kinesis.putRecord(...)` )를 잔뜩 짜두었고, S3로 자동 백업되는 Firehose까지 예쁘게 연결해 두었습니다. 그런데 요금이 너무 나와서 구글 클라우드로 이사가고 싶습니다.
- **결과: 대참사가 일어납니다.** 구글 클라우드에는 Kinesis도 없고 Firehose도 없습니다.
  1. 백엔드 서버에 짜둔 Kinesis 전용 전송 코드를 다 지우고, 구글 클라우드 전용(Pub/Sub)으로 코드를 **전부 다시 짜야 합니다.**
  2. 클릭 한 번으로 S3에 백업하던 Firehose가 없으니, 백업 파이프라인을 **처음부터 새로 설계하고 구축해야 합니다.** ---

### 💡 결론: 껌데기만 빌리느냐, 뼈대까지 빌리느냐

클라우드 회사(AWS, 구글, MS)들은 고객이 자기를 서비스를 영원히 쓰게 만들고 싶어 합니다. 그래서 웹 서버 같은 단순한 '가상 컴퓨터'를 넘어서, Kinesis, DynamoDB, Lambda 같은 '자기를 클라우드에서만 돌아가는 압도적으로 편한 독점 서비스'를 미친 듯이 만들어냅니다.

편리함을 취할수록 AWS라는 늪에 깊게 빠지게(Lock-in) 되고, 나중에 빠져나오려면(마이그레이션) 수개월의 시간과 수억 원의 개발 비용을 치러야 합니다.

그래서 실무에서는 핵심 파이프라인과 인프라(Kafka, DB 등)를 특정 클라우드에 종속되지 않게 설계하는 '**클라우드 아그노스틱(Cloud-Agnostic)**' 전략을 매우 중요하게 생각합니다. 고객님의 질문이 바로 이 거대한 IT 산업의 딜레마를 정확히 관통한 것입니다!

- 실무에서는 "웹 서버는 AWS, 데이터 분석과 AI는 구글 클라우드(GCP), 사내 보안 데이터는 자체 서버(On-premise)"를 섞어 쓰는 **멀티 클라우드(Multi-Cloud)** 전략을 많이 취합니다.
- **Kafka는 오픈소스**이기 때문에 AWS, GCP, 자체 서버 등 어디서든 똑같이 띄워서 쓸 수 있습니다. 회사가 AWS에서 다른 클라우드로 이사를 가더라도 파이프라인을 그대로 들고 갈 수 있는 강력한 자유도가 있습니다.

→ 2번의 이유로 **AWS Kinesis 탈락**

## 3. 세밀한 튜닝과 한계 돌파 (Customization)

Kinesis는 AWS가 관리해 주므로 편하지만, 반대로 말하면 '블랙박스'입니다. AWS가 정해놓은 한계치를 마음대로 바꿀 수 없습니다.

- **메시지 크기 제한:** Kinesis는 한 번에 보낼 수 있는 데이터 크기가 1MB로 고정되어 있습니다. 더 큰 데이터를 통째로 넣고 싶어도 불가능합니다. (Kafka는 설정으로 한도 해제 가능)
- **보관 기간:** Kinesis는 데이터를 최대 1년까지만 보관할 수 있습니다. (Kafka는 하드디스크가 허락하는 한 무제한 보관 가능)
- **성능 최적화:** 초당 수백만 건의 트래픽을 처리하려면 네트워크 스레드 수, 디스크 I/O 방식 등을 영혼까지 끌어모아 튜닝해야 하는데, Kinesis는 이런 세부 설정이 불가능하고 Kafka는 100% 통제할 수 있습니다.

## 4. 압도적인 생태계 (Ecosystem & Community)

Kafka는 링크드인(LinkedIn)에서 만들어져 오픈소스로 풀린 지 10년이 넘었습니다.

그동안 전 세계 수많은 천재 개발자들이 Kafka에 붙여 쓸 수 있는 '플러그인'들을 다 만들어 두었습니다.

- DB 데이터를 실시간으로 퍼오는 **Kafka Connect**, 실시간 데이터 가공에 특화된 **Kafka Streams**, SQL로 실시간 스트림을 분석하는 **ksqlDB** 등 생태계가 너무나도 방대합니다.
- Kinesis도 훌륭하지만, 이미 시장을 장악한 Kafka의 방대한 외부 연동(Third-party) 생태계를 따라잡기는 어렵습니다.

### 요약하자면

- **Kinesis**는 다 알아서 해줌 + 비쌈
  - **Kafka**는 직접 운영하고 조립해야함. 맘대로 성능 뽑아낼 수 있음 kinesis 대비 유지비 싼"
- 
- 운영 경험, 비용 절감, 선택할 수 있는 시스템의 다양성 → kafka 사용